

AI + 科研工作流程经验分享

1 小时逐字讲稿

建议时长：约 60 分钟。括号中的“操作提示”仅供现场切换页面、点开视频或图片时参考，不需要念出。

各位老师、同学大家好，我是刘洋，来自四川大学计算机学院。今天我想和大家分享的主题是“AI + 科研 workflow”。

先说明一下，今天这个分享不是想讲一个特别宏大的概念，也不是想说 AI 已经可以替代科研人员了。恰恰相反，我今天想表达的核心非常简单：AI 可以帮我们做很多执行层面的工作，但真正的判断、选择和核验，仍然要由人来完成。

所以这一页我放了一句话：AI 负责执行，人负责判断。这个也是我最近一段时间使用 AI 工具后最深的感受。

以前我们说 AI 辅助科研，很多时候想到的是让它帮忙翻译、润色、总结文献。这个当然有用，但还只是比较浅的一层。现在更值得关注的是：我们能不能把科研里面很多重复、繁琐、低效率的环节，重新组织成一个更顺畅的工作流？比如文献阅读、资料整理、代码复现、图表生成、报告撰写、网页展示，这些事情本来是分散的，但现在可以通过 AI 工具串起来。

我今天的分享就围绕这条主线展开：用 AI 工具重组科研 workflow。

今天大概分成几部分。第一部分，我先讲不同 AI 交互入口应该怎么选。第二部分，讲模型怎么搭配。第三部分，讲 Vibe Coding，也就是用自然语言驱动代码生成和迭代。第四部分，讲 Vibe Research，也就是用 AI 协助科研任务流程化。最后我会展示三个真实案例：专利材料整理、科研绘图和学术润色、以及论文算例复现。

我不会把重点放在“某一个工具有多厉害”，而是想让大家看到：面对一个真实任务，我们应该怎么判断入口、怎么写提示词、怎么验收结果，以及如何避免被 AI 带偏。

（操作提示：点击 *START* 或切到“AI 交互入口怎么选？”页面。）

二、AI 交互入口怎么选（约 8 分钟）

这一页我想先解决一个很实际的问题：现在 AI 工具太多了，ChatGPT、Kimi、豆包、Gemini、Codex、Trae、Cursor、VSCode、Claude Code、Codex CLI，很多同学刚接触的时候会很混乱。到底哪些是聊天工具，哪些是 Agent，哪些是 IDE，哪些是命令行工具？它们不是一个维度的东西。

我觉得最实用的分类不是按厂商分，也不是按模型分，而是按“交互入口”来分。也就是：你是在哪里和 AI 交互？你把什么样的任务交给它？它能不能读文件、改代码、跑命令、管理工程目录？

第一类是聊天框。

聊天框是最常见的入口，比如 ChatGPT、Kimi、豆包、Gemini。它的核心价值不是“替你完成整个项目”，而是帮你把问题想清楚，把材料读明白，把表达改顺。

比如我读一篇论文，看不懂创新点，我可以把摘要、引言、方法部分给它，让它帮我解释这篇论文到底解决了什么问题。比如我写了一段中文结果描述，感觉不够学术，可以让它改成英文。再比如我需要给老师汇报，可以让它帮我整理一个讲述逻辑。

聊天框的优势是门槛最低，打开就能用；而且它非常适合快速问答、概念解释、论文阅读、翻译润色、总结归纳、生成思路和提示词。

但是它的边界也很明显。它主要靠对话和复制粘贴，不适合长期管理复杂文件，也不适合反复执行一套流程。比如你有三十多个 PDF、好几个代码文件、一堆输出表格，这些东西如果全部靠复制粘贴给聊天框，就会很乱。上下文一长，也容易断。

所以聊天框适合“理解问题”和“整理表达”，但不一定适合“完整交付”。

第二类是 AI Agent。

这里我把它叫做任务交付入口。比如 Codex App、Trae Solo 或 Work、小龙虾这一类工具。它们和聊天框最大的区别是：我不是只问它一个问题，而是把一个完整目标交给它。

比如我可以说：这里有一个论文 PDF 文件夹，请你定位里面的算例，用 Python 复现，运行代码，生成图表和报告。它会自己读材料、拆步骤、改文件、运行验证，最后把结果交付出来。

Agent 更像“接任务的人”。我给它目标、材料和验收标准，它自己组织执行过程。我不一定逐行盯着它怎么写代码，但我一定要看最终产物是否达标。

所以 Agent 适合什么？适合专利整理、论文复现、批量处理、自动修错、生成报告、生成网页 Demo。这些任务的共同特点是：不是一句话问答，而是有输入、有过程、有输出、有验收。

但 Agent 也不是万能的。它很容易在任务不清楚的时候自作主张，所以我们给 Agent 任务时，必须明确边界：输入是什么，输出是什么，不允许编造什么，哪些地方必须人工核验。

第三类是 AI 编程 IDE。

这个和 Agent 工作台不完全一样。比如 Trae、Cursor、VSCode 加 Copilot 这一类，我更愿意把它们理解成“把 AI 放进编辑器”。

AI 编程 IDE 的优势不是完全托管，而是边写边审。它能看文件树、看 diff、改局部代码、跑终端、连接远程服务器；我可以随时人工接管，看到它到底改了哪个文件、改了哪几行。

这类工具适合真实项目迭代，比如我要修改一个网页界面，要保留原有风格但调整排版；或者我要改一个 Python 项目，只动其中某几个函数；或者我要连服务器调环境、跑脚本。这种时候，AI 编程 IDE 比纯 Agent 更适合，因为它给我更强的可见性和控制权。

简单说，Agent 偏“交任务等结果”，AI IDE 偏“边改边看边确认”。

第四类是 CLI Agent，也就是命令行 Agent。

比如 Codex CLI、Claude Code、iFlow 这类。很多人一听 CLI 会觉得很硬核，好像是为了炫技。其实不是。CLI 的价值在于，它最贴近真实运行环境。

科研里很多任务并不是在浏览器里完成的，而是在 Linux 服务器、远程环境、无图形界面、SSH、批处理脚本里完成的。比如我要安装依赖、跑训练、看日志、改 Python 脚本、处理大量文件，这时 CLI Agent 就很有优势。它可以直接进入终端和工程目录，读文件、改代码、跑命令、看日志。

但 CLI 的风险也更高。因为它离真实文件系统更近，删除、覆盖、联网、权限操作都要谨慎确认。所以 CLI 适合有一定基础之后再用，尤其适合服务器场景。

总结这一页，我想让大家记住的不是某个工具名字，而是四种入口的分工：聊天框负责理解问题；Agent 负责完整任务交付；AI IDE 负责真实项目的编辑和审阅；CLI 负责服务器和真实运行环境。

（操作提示：可以依次点开四个“大屏预览”，每个视频只展示 10 到 20 秒，强调交互入口差异即可。）

三、模型怎么选（约 7 分钟）

讲完入口之后，再来看模型。

很多人刚开始用 AI 的时候，会问一个问题：到底哪个模型最强？我应该只用哪一个？

我的建议是，不要迷信一个模型通吃。不同模型适合不同任务，把复杂推理、长文献、多模态、低成本 API 分开搭配，体验会更稳。

这一页我把模型分成海外模型和国产模型两类。

先说海外模型。ChatGPT 我目前会把它作为复杂问题、新项目规划、代码实现和写作润色的主力。它的综合能力比较强，尤其适合我从零规划一个任务：比如一个科研展示网页怎么组织，论文复现流程怎么拆，代码项目出了问题怎么定位。

Gemini 的优势更多在多模态和资料阅读，特别是和 NotebookLM 搭配时，处理课程材料、论文资料包、长文档会比较方便。它不一定每个细节都比其他模型强，但作为资料库阅读入口，是一个很好的补充。

Claude 在长文本理解、代码理解和写作风格上也有特点，不过对我们很多人来说，它的账号、支付和验证门槛相对更高，所以我会把它作为观察和补充，而不是默认主力。

再看国产模型。

Kimi 我觉得非常适合长上下文和中文文献调研。比如你有一篇很长的论文、一份报告，或者要做中文材料归纳，Kimi 的体验是比较顺的。

豆包的优势是手机端和桌面端都方便，多模态、拍照问答、日常资料整理非常顺手。比如我现场拍一个图、拍一页材料，让它解释一下；或者语音输入一个想法，让它帮我整理成文字，这种低门槛场景很适合豆包。

DeepSeek 的特点是推理、代码和 API 成本有优势。对个人来说，聊天端可以直接用；对自动化脚本、Agent 或批处理来说，低成本 API 很有价值。很多时候不是每个任务都需要最贵的模型，简单任务用性价比高的模型就够了。

通义千问、文心一言、腾讯元宝、智谱清言、讯飞星火这些模型，也各有生态和场景。比如阿里系有云端 API 和开源生态，百度系在搜索和办公场景比较强，腾讯元宝和微信生态衔接方便，讯飞星火在语音、教育和办公场景有积累。

但这里我不建议大家陷入模型排行榜。模型能力变化很快，今天强的模型，过几个月可能就不是第一了。更稳定的思路是：根据任务类型搭组合。

我自己的推荐组合是：复杂问题用 GPT；长文献用 Kimi；多模态随手用豆包；低成本 API 用 DeepSeek；资料库阅读可以配 Gemini 或 NotebookLM。

也就是说，模型不是信仰，而是工具箱里的不同扳手。科研工作流真正要解决的是：这个任务现在需要理解、需要写代码、需要读长文档、需要看图，还是需要低成本批量执行？先判断任务，再选模型。

（操作提示：这里可以快速扫过模型卡片，不需要逐一展开价格细节。强调“按任务搭配”，不要停留太久。）

四、Vibe Coding：用自然语言驱动代码迭代（约 10 分钟）

接下来进入 Vibe Coding。

Vibe Coding 这个词最近比较火。它的核心不是说完全不懂代码也能做大型工程，而是说我们可以用自然语言描述开发目标，让 AI 生成、修改和调试代码；人负责说清需求、运行检查和验收结果。

这里我特意把页面上的词写成：低代码门槛，一句话起步，多轮迭代，先跑通再优化。

这几个词很重要。

第一，低代码门槛，不等于零门槛。你不一定要每一行代码都自己写，但你至少要知道结果对不对、功能能不能跑、哪里不符合预期。

第二，一句话起步，不等于一句话结束。很多任务确实可以从一句话开始，比如“帮我做一个番茄钟网页小程序”。但第一版通常只是初稿，后面一定要继续反馈、修改和优化。

第三，多轮迭代才是关键。以前我们写代码，是自己写、自己报错、自己查资料、自己改。现在可以把这个过程变成：提出需求，AI 生成，运行体验，判断是否达标。如果达标就交付；如果不达标，就把问题反馈给 AI，再继续生成。

所以 Vibe Coding 的闭环不是“让 AI 一次写对”，而是“先跑通，再根据效果、报错和验收标准继续迭代”。

这里我用番茄钟网页小程序做了一个很简单的例子。

第一条提示词是：请帮我做一个番茄钟网页小程序，包含番茄钟、短休息、长休息三种模式，支持开始、暂停、重置和任务管理。请用单个 HTML 文件实现，界面简洁、可直接在浏览器打开。

这句话其实已经包含了几个关键信息。第一，任务目标是番茄钟网页小程序。第二，功能范围包括三种模式、开始、暂停、重置、任务管理。第三，交付形式是单个 HTML 文件。第四，验收方式是能直接在浏览器打开。

这就是一个比较好的起手提示词。它不长，但把任务边界讲清楚了。

然后第一版出来之后，我不满意界面，就进入第二轮迭代。我给它参考图，再说：请参考它的整体视觉风格重做一版，保留红色背景、半透明圆角面板、顶部三段切换、大号圆形倒计时、开始和重置按钮、任务管理区域，整体尽量接近参考图。

这个时候 AI 就不只是写功能，而是在做视觉复刻和交互优化。

从这个例子可以看出，Vibe Coding 真正改变的是开发方式。以前我可能要先学 HTML、CSS、JavaScript，再慢慢写。现在我可以先把想法说出来，快速得到一个可运行版本，再通过反馈不断接近想要的效果。

但是这里有一个很重要的边界：越是重要的代码，越不能只看“它生成了”。一定要运行、测试、检查。比如按钮能不能正常工作，计时逻辑是否正确，刷新页面会不会丢数据，移动端显示是否正常。这些都是人的验收工作。

如果把这个思路放到科研里，意义就更大了。很多科研代码并不一定是大型软件工程，而是为了处理数据、画图、复现实验、生成报告。对这类任务来说，Vibe Coding 可以显著降低起步成本。

但我们不能把它理解成“AI 替我科研”。更准确的说法是：AI 帮我更快地把想法变成可运行的东西，而我负责判断这个东西是否可信、是否符合任务目标。

（操作提示：可以展示两个 PROMPT，点开番茄钟参考图，说明“第一版跑通功能，第二版根据视觉参考继续迭代”。）

五、Vibe Research：从“会问 AI”到“会组织科研流程”（约 13 分钟）

接下来是 Vibe Research。

如果说 Vibe Coding 是用自然语言驱动代码生成和迭代，那么 Vibe Research 就是把类似的思路放到科研任务里。

我在这一页写了三句话：人定方向，AI 执行，人做核验。

这句话是整个分享里最重要的一句。因为科研不是简单生成内容，科研最核心的是判断：问题是否成立，证据是否充分，方法是否合理，结论是否可信。这些不能交给 AI 无脑决定。

但科研里也有大量执行层面的工作，非常适合 AI 协助。比如读论文、整理文献、翻译润色、提取信息、生成图表、写代码、跑脚本、生成报告。这些工作如果完全手工做，会耗费大量时间。

所以 Vibe Research 的目标不是让 AI 替代科研判断，而是把科研任务拆成可描述、可执行、可检查、可复用的流程。

这一页我把 Vibe Research 分成两条路径。

第一条是聊天框路径。

聊天框路径就是用提示词或模板，让 ChatGPT、Kimi、豆包等完成阅读、总结、翻译、润色和方案讨论。

这里又可以分成两种：提示词和模板。

提示词适合临时问题。比如我今天读一篇论文，看不懂创新点，就可以问：“请解释这篇论文的创新点，它和已有方法相比解决了什么问题？”比如我写了一段结果描述，可以问：“请把这段结果描述改成学术英语，保持原意，不要夸大。”再比如我准备汇报，可以问：“根据我的研究方向，帮我设计一个汇报提纲。”

模板适合高频任务。比如读论文，我每次都想知道研究背景、核心问题、方法创新、实验设计、主要结果、局限性和可复现性，那我就可以把这个流程固定成一个模板。以后每读一篇论文，就直接用这个模板。

同样，文献综述、审稿意见回复、图注润色、开题汇报、组会汇报，都可以做成模板。模板的价值是减少每次重新组织语言的成本，也让输出更稳定。

聊天框路径的优势很明显：打开就能用，学习成本低；适合快速理解和头脑风暴；中文表达、润色、翻译都很方便。

但它的不足也明显：主要靠复制粘贴，难管理文件；复杂任务容易断上下文；不能稳定复现执行过程。所以聊天框路径适合“快速理解”和“规范表达”，但不适合“长流程自动执行”。

第二条是 **Agent** 路径。

Agent 路径更进一步。它不是只回答问题，而是可以读取文件、调用工具、执行代码、生成图表和报告。

比如我给它一个论文文件夹，它可以定位算例，写 **Python** 代码，运行报错后自动修改，最后生成误差表、图表和 **HTML** 报告。这就不是聊天框能轻松完成的任务。

Agent 路径里还有一个很重要的概念，叫 **Skills**，也就是技能。

什么时候需要 **Skills**？当一个任务可以写成固定步骤，就适合封装成技能。比如什么时候调用，读哪些材料，按什么规范输出，哪些地方必须核验，这些都可以写进技能里。这样下次遇到类似任务，**Agent** 就不需要每次重新理解规则，而是直接调用流程。

这里我拿 **Nature-Skills** 做例子。

Nature-Skills 可以理解为一组面向科研场景的 **Agent** 技能包。它不是普通提示词合集，而是把“读材料、按规范执行、生成图文产物、保留核验点”整理成智能体可以调用的流程。

比如 **nature-figure**，面向科研图工作流，可以帮助生成更规范的科研图表。**nature-polishing**，可以把学术文本润色、重构或翻译成更规范的英文表达。**nature-literature-pipeline**，可以做自动化文献发现、评分、精读推送和本地归档。**nature-reader**，可以生成带来源锚点、图文对应和中英文对照的全文 **Markdown reader**。

这些技能背后的逻辑其实很简单：很多工具都可以被提炼成标准化流程，而标准化流程就可以封装成可复用技能。

这对科研很有启发。

比如我们平时做文献阅读，其实不是每次都随机读。我们会先看标题摘要，再看方法，再看实验，再看结论，再判断能不能复现。这个流程可以标准化。

比如我们做科研绘图，也不是每次都从零开始。我们需要统一字体、坐标轴、误差条、图例、配色、分辨率、导出格式。这也可以标准化。

比如我们做论文复现，也有流程：先定位算例，提取参数，理解方程，写代码，跑结果，比较误差，调整偏差，生成报告。这个流程也可以标准化。

所以 Vibe Research 的核心不是“我会写一个很长的提示词”，而是“我会把科研任务拆成流程，并且知道哪些地方必须人工核验”。

最后总结一下这一页：先用聊天框快速理解；再用模板规范输出；复杂任务交给 **Agent**；稳定流程封装成 **Skills**；最终结论必须人工核验。

这也是我今天想传达的科研 AI 使用观：AI 可以提高执行效率，但科研可信度来自人的核验。

（操作提示：可以点开 [Nature-Skills GitHub](#)，简单展示项目；点击“查看更多技能”弹窗，说明不同技能的功能，不需要逐个展开太久。）

六、Demo 01：微信需求到专利目录（约 7 分钟）

下面进入第一个真实案例：专利材料整理。

这个案例的来源很普通，就是一个微信里的真实需求。对方发来一个压缩包，说能不能帮忙把专利按照参考格式排个序，像 Word 目录一样整理成 TXT 文档。还特别说明：按照授权和申请分一下，授权和申请看右上角，只用看第一页。

这个任务听起来不复杂，但如果手工做，其实很烦。

因为有一批专利 PDF，要一个个打开，看第一页右上角，判断它是授权还是申请。然后还要提取专利名称、编号、日期，再按照授权和申请分组，每组按时间降序排列，最后做成 TXT 目录。数量一多，就很容易抄错、漏掉、排序错。

这类任务很适合 **Agent**。因为它有明确输入：专利 PDF 文件夹和格式参考图。也有明确规则：只看第一页右上角，区分授权和申请。也有明确输出：TXT 目录和 Excel 或 CSV 核对表。还有明确边界：不确定就标待核验，不要编造。

这里我把提示词故意写得很短，因为我想展示低使用门槛。不是说一定要写一大段复杂 prompt 才能用 AI。只要任务边界说清楚，短提示词也可以很有效。

我的提示词是：我会给你一个专利 PDF 文件夹和一张格式参考图。请按图整理成 TXT 目录：已授权和已申请分开；每个 PDF 只看第一页右上角；提取专利名称、编号和日期；每类按日期降序。再生成一份 Excel 或 CSV 核对表。不确定就标“待核验”，不要编造。

这段话包含了五个关键信息。

第一，输入材料是什么：PDF 文件夹和格式参考图。

第二，处理规则是什么：已授权和已申请分开，只看第一页右上角。

第三，要提取什么字段：专利名称、编号、日期。

第四，排序规则是什么：每类按日期降序。

第五，交付和边界是什么：生成 TXT 目录和核对表，不确定标待核验，不编造。

这就是一个比较适合 **Agent** 的任务描述。它不长，但能验收。

这里我还建议输出核对表。为什么？因为 **AI** 处理 **PDF** 时可能会识别错，尤其是扫描件或者图片质量不高的 **PDF**。如果只给我一个 **TXT**，我很难快速检查。核对表里保留来源文件、类型、名称、编号、日期和待核验项，我就可以快速人工复核。

所以这个案例想说明的是：**AI** 不只是写作工具，它可以处理现实里的杂活。很多科研行政和成果整理任务，本质都是“从一堆材料中按规则提取信息”，这类任务非常适合交给 **Agent**。

但注意，最后一定要人工核验。尤其是专利号、日期、授权公告号、申请公布号，这些字段不能错。**AI** 可以帮我们加速整理，但不能替我们承担准确性责任。

（操作提示：点击左侧微信截图放大，说明真实任务来源；再展示提示词和右侧输出目标。）

七、Demo 02：科研绘图与学术润色（约 5 分钟）

第二个案例是 **Nature-Skills** 的实战演示：科研绘图和学术润色。

这个案例想解决的痛点是：数据整理、绘图、结果描述和英文润色经常被割裂处理。

我们平时做科研时，可能先用 **Python** 或 **Origin** 画图，然后再手写一段中文结果描述，再复制到另一个工具里翻译成英文，再反复改语言。每一步都能做，但整体很碎，效率不高。

这里我把流程拆成四步。

第一步，准备数据。让 **AI** 生成一份模拟实验数据并保存为 **CSV**。这里当然只是演示，真实科研中数据应该来自实验或仿真结果，不应该让 **AI** 编造真实数据。

第二步，画图。调用 **nature-figure**，基于 **CSV** 数据生成科研风格图表。这里的重点不是“画出一张图”这么简单，而是尽量让图表符合科研表达习惯，比如坐标轴、图例、字体、分辨率、输出格式都要规范。

第三步，写结果。根据图表结果，生成一段简洁、准确的中文结果描述。这里可以让 **AI** 先帮我们把现象说清楚，比如哪个指标上升，哪个指标下降，趋势是否明显。

第四步，润色。调用 **nature-polishing**，把中文结果描述润色成规范的学术英语。

这四步加起来，其实就是一个小型科研写作 workflow：数据到图表，图表到结果描述，中文到英文，最后人工检查。

这里同样要强调边界。AI 可以帮我们表达图中的趋势，但不能替我们确认这个趋势是否具有统计意义，也不能替我们判断实验设计是否合理。尤其是真实论文里的结果描述，不能为了好看而夸大。

所以这个案例想说明：当一个科研任务是重复出现的，比如绘图、图注、结果描述、英文润色，我们就可以把它流程化，让 Agent 或 Skills 帮我们减少重复劳动。

（操作提示：播放 demo_02 录屏，展示从提示词到图表再到英文润色的过程。）

八、Demo 03：从论文到复现结果（约 7 分钟）

第三个案例是论文复现。

这也是最能体现 Vibe Coding 加 Vibe Research 的案例。因为它既需要读论文，又需要写代码，还需要生成误差表、收敛阶和图像，并且要以论文结果为基准进行对比。

这里我放了一个作者沟通记录的微信截图。这个截图想表达的是：这不是我凭空设计的玩具任务，而是一个真实的科研交流场景。作者反馈中提到，有些事情如果靠自己慢慢做可能要很久，但用 AI 后速度明显提高。这个背景能说明，论文复现这种任务确实有实际意义。

在这个案例里，任务输入是论文 PDF 和算例编号，输出是误差表、收敛阶和图一，验收标准是以论文结果为基准。

我给 Agent 的现场提示词是：请认真理解这篇论文中的内容，然后用 Python 复现 PDF 中算例 4.1，完整生成误差表、收敛阶和图一，结果以论文为基准，允许合理数值误差，但要尽量保持一致，若有偏差明显则需要继续调试。

这段提示词比前面的专利整理稍微复杂一点，因为论文复现本身更复杂。但它仍然是自然语言，没有写一堆公式。

里面有几个关键点。

第一，“认真理解论文内容”。这要求 Agent 先读材料，而不是直接猜。

第二，“用 Python 复现算例 4.1”。这明确了任务范围，不是复现整篇论文。

第三，“完整生成误差表、收敛阶和图一”。这明确了交付物。

第四，“结果以论文为基准，允许合理数值误差，但要尽量保持一致”。这句话很重要，因为数值计算中完全一模一样不一定现实，可能和机器精度、实现细节、网格参数有关。但如果偏差明显，就不能直接交差。

第五，“若有偏差明显则需要继续调试”。这让 Agent 不只是生成一次，而是进入迭代。

这个案例的流程可以分成四步：读 PDF，定位算例 4.1；写代码，用 Python 实现复现流程；对结果，生成误差表、收敛阶和图一；调偏差，如果和论文不一致，就继续检查参数、边界条件、误差定义和数值格式。

这类任务最容易出现的问题是“表面复现”。也就是说，图看起来有点像，但公式、边界条件或者误差定义其实不一样。所以人工验收非常重要。我们要看代码是否对应论文方法，参数是否对应原文，误差表是否合理，收敛阶是否符合理论或论文结果。

但即便如此，AI 仍然能显著降低起步难度。以前我想复现一篇不熟悉领域的论文，可能第一步就卡在公式和代码结构上。现在我可以先让 Agent 搭一个可运行框架，再逐步检查和修正。

所以这个案例想说明：Vibe Research 不是只做文字整理，它可以深入到科研计算流程。但越深入，越要强调验收，越不能无脑相信输出。

（操作提示：点击左侧聊天截图放大；播放右侧论文复现录屏，停在误差表或图像结果处说明验收标准。）

九、科研工具箱与收尾（约 5 分钟）

最后这一页是科研工具箱。

我把它做成一个小彩蛋，里面列了一些我平时会用到的网站、插件和软件。这里不是要大家一次性全部装上，而是想说明：AI + 科研不是只靠一个大模型，更多时候是模型、工具和人的判断组合在一起。

网站工具里，比如图片压缩可以处理报告和网页图片体积；Doc2X 可以辅助 PDF 或文档解析；YouMind 可以辅助提示词；素材解析可以帮忙处理展示素材；硅基流动可以作为 API 接入入口。

插件里，比如 Easy Scholar 可以辅助查看期刊和论文信息；iTab 可以管理浏览器入口；AdGuard 用来减少网页干扰；沉浸式翻译适合读英文网页；Zotero Connector 则适合文献收藏。

软件里，Typora 适合写 Markdown；Zotero 适合管理文献；豆包桌面端适合日常多模态和轻量任务；Codex 和 Trae 更偏 Agent 和 IDE 工作流。

但工具只是工具，不能本末倒置。真正重要的是最后这句话：模型负责理解，工具负责执行，人负责判断。

科研中的关键结论、数据可信度、公式推导和事实边界，必须由人来最终核验。

今天我想传达的主要不是“大家一定要用某个工具”，而是希望大家建立一种工作流意识。

遇到一个科研任务时，我们可以先问自己几个问题：这个任务是需要理解，还是需要执行？是一次性问题，还是高频任务？输入材料是什么？输出结果是什么？怎么验收？哪些地方不能让 AI 编造？哪些地方必须人工复核？

如果只是临时理解和表达，用聊天框就够了。如果是高频写作和阅读任务，可以做成模板。如果是多步骤项目，可以交给 **Agent**。如果流程稳定，就可以封装成 **Skills**。如果涉及真实代码和服务端，就用 **IDE** 或 **CLI**，并且谨慎确认修改和命令。

我自己的经验是，AI 最有价值的地方，不是帮我省掉思考，而是帮我把重复劳动、格式整理、初步代码和报告生成这些执行层面的工作压缩掉。这样人可以把更多精力放在真正需要判断的地方：问题值不值得做，方法是否合理，结果是否可信。

最后再回到开场那句话：**AI** 负责执行，人负责判断。希望今天这个分享能给大家一个比较直观的参考，让大家在自己的科研学习中，把 **AI** 用得更稳、更具体，也更可控。

我的分享就到这里，谢谢大家。接下来如果大家有问题，或者对某个案例想看更具体的操作，我们可以一起交流。

十、可能的问答补充稿（可选，约 5 到 10 分钟）

如果有人问：完全不懂代码能不能做 **Vibe Coding**？

可以这样回答：可以做简单 **demo**，也可以快速起步，但不建议完全不懂就做高风险任务。因为 AI 生成代码之后，人至少要能运行、看报错、判断功能是否符合需求。低代码门槛不等于没有验收门槛。

如果有人问：**AI** 整理的文献和结论可靠吗？

可以这样回答：不能直接当最终结论。**AI** 适合做初筛、总结和结构化整理，但关键文献、关键数据、关键结论必须回到原文核验。尤其是引用、公式、数值和作者观点，不能只看 **AI** 的概括。

如果有人问：科研里最适合先用 **AI** 做什么？

可以这样回答：我建议从低风险、高重复的任务开始，比如论文摘要总结、英文润色、图注改写、文件批量重命名、PDF 信息提取、数据清洗脚本、绘图脚本和汇报材料初稿。这些任务即使 **AI** 做错，也比较容易检查和修正。

如果有人问：**Agent** 最大的风险是什么？

可以这样回答：最大的风险是任务边界不清。你没说清楚输入、输出、禁止事项和验收标准，它就可能自作主张。所以我现在给 **Agent** 任务时，通常会写清楚材料在哪里、要输出什么格式、不确定怎么办、哪些地方不能编造。

如果有人问：怎么判断该用聊天框、Agent、IDE 还是 CLI？

可以这样回答：我会按任务粒度判断。只是问问题、改文字，用聊天框。要完整交付一个文件夹项目，用 Agent。要在真实项目里边改边看，用 AI 编程 IDE。要在服务器和命令行环境里跑脚本、看日志，用 CLI。

如果有人问：AI 会不会让我们更懒？

可以这样回答：如果只是复制粘贴答案，确实会。但如果把 AI 当执行助手，把自己放在提出问题、设计流程、检查结果的位置，它反而会逼着我们更清楚地定义任务和验收标准。关键在于，人不能放弃判断。